Network Traffic Analysis Using Wireshark

Introduction

Network Traffic Analysis is the process of capturing and examining data packets that travel across a network. Every activity performed on a system such as browsing a website, resolving a domain name, or checking connectivity generates network traffic. By analyzing this traffic, we can understand how different network protocols operate and how data is transmitted between systems.

In this project, **Wireshark** is used to capture live network traffic and analyze commonly used protocols such as **TCP, UDP, ICMP, HTTP, HTTPS, and DNS**. This analysis helps in understanding packet structure, communication flow, and protocol behavior in real-time.

Objective of the Experiment :

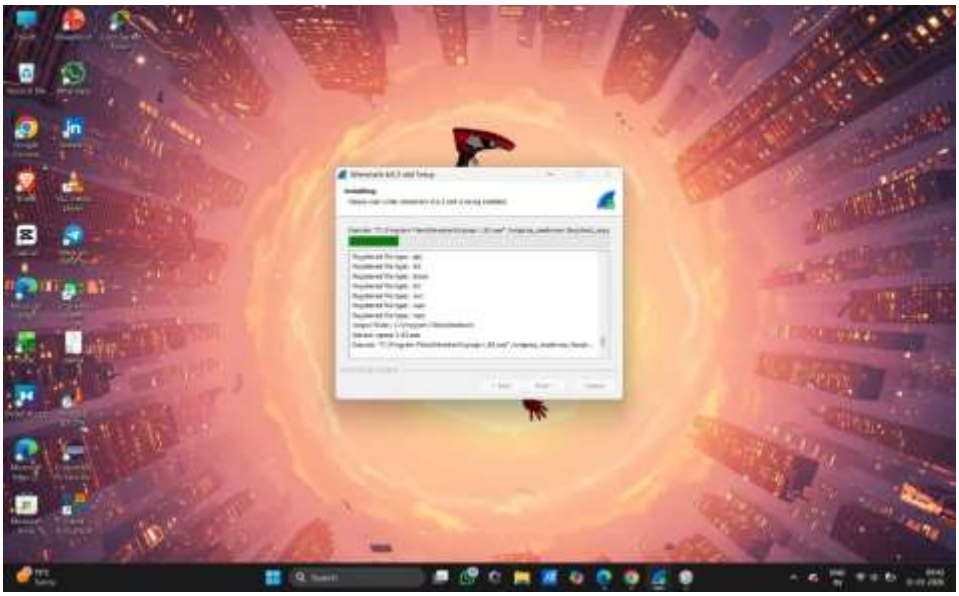
- To capture live network packets using Wireshark
- To identify and analyze common network protocols
- To understand how source and destination IP addresses are used
- To analyze ports, packet size, and protocol-specific flags
- To observe the differences between connection-oriented and connectionless protocols

Tools and Requirements Software Required :

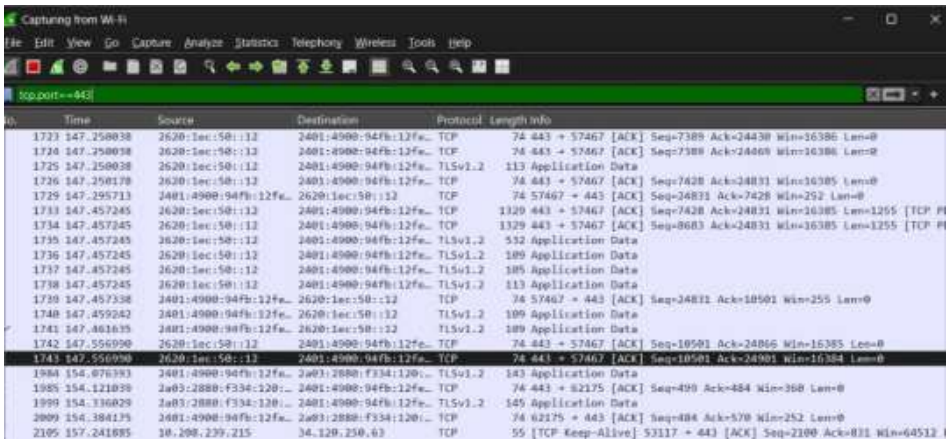
- ✓ Wireshark – for packet capture and analysis
- ✓ Web Browser to generate HTTP/HTTPS traffic
- ✓ Command Prompt / Terminal to generate DNS and ICMP traffic
- ✓ System Requirements Operating System: win / linux
- ✓ Active Internet connection
- ✓ Wireshark can be downloaded from the official website: https://www.wireshark.org/download.html

Wireshark Configuration and Setup

- ✓ Install Wireshark on the system.
- ✓ Launch Wireshark with appropriate permissions.
- ✓ Identify the active network interface (Wi-Fi or Ethernet).



- ✓ Double-click on the active interface or select it and click the Start Capture (▶) button.
- ✓ Wireshark will now begin capturing live network traffic.



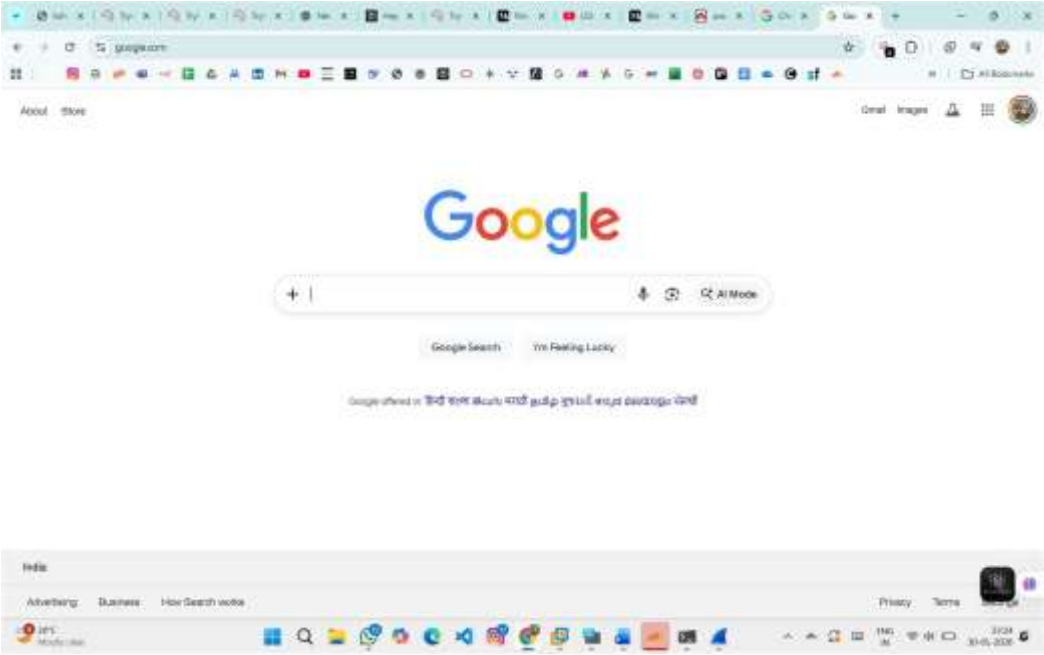
- ✓ At this point, the packet capture session has started.

Generating Network Traffic

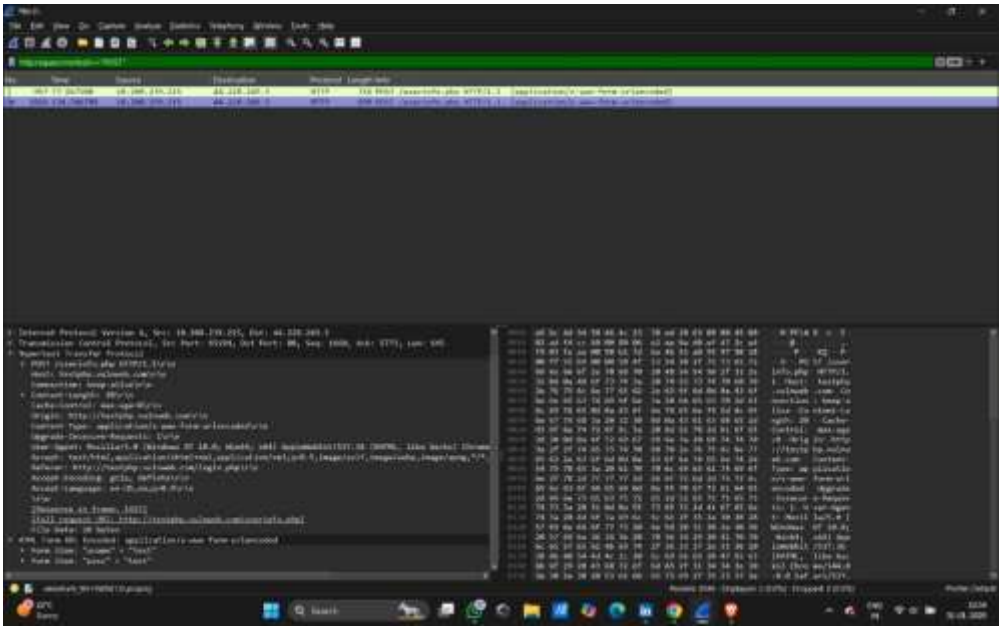
To analyze different protocols, traffic must be generated manually.

Perform the following actions during packet capture:

- Open a web browser and visit: https://google.com



- This action generates HTTP/HTTPS, DNS, TCP, and TLS traffic.
- After completing the activity, stop the packet capture by clicking the red Stop button (■) in Wireshark.

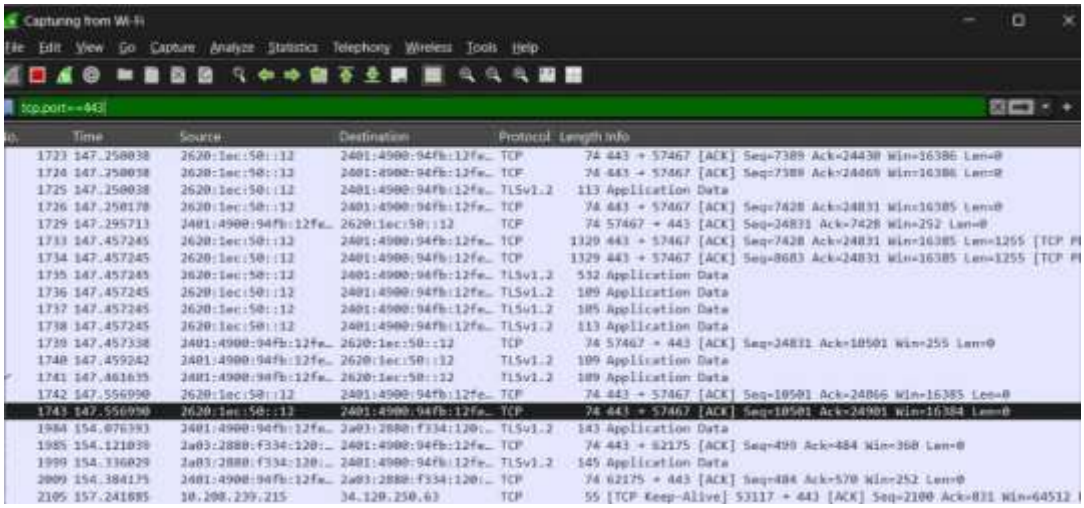


Observations and Output Analysis

1. TCP Traffic Analysis

Display Filter Used

Tcp



Observation

- The Protocol column displays TCP packets.

- Source IP address corresponds to the local system.
- Destination IP address corresponds to the remote server.
- Source Port is a randomly assigned ephemeral port.
- Destination Port is usually:
  - Port 80 (HTTP)
  - Port 443 (HTTPS)

TCP Flags Observed

- SYN – Connection request from client
- SYN-ACK – Server acknowledgment
- ACK – Final acknowledgment from client

Output and Explanation

TCP establishes a connection using a three-way handshake mechanism. This process ensures reliable communication before data transfer begins. TCP guarantees packet delivery, correct order, and error checking, which makes it suitable for web communication and file transfers.

2. UDP Traffic Analysis

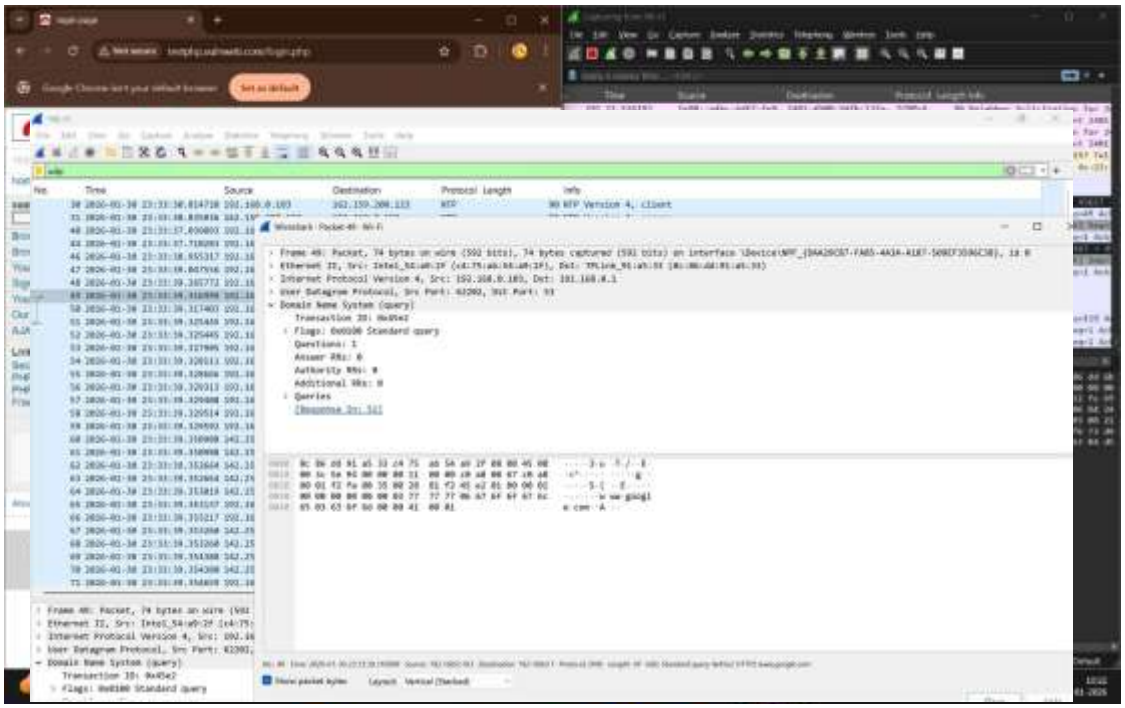
How to Generate UDP Traffic

1. Start Wireshark capture.
2. Open Command Prompt (Windows).
3. Execute the command:
4. nslookup google.com
5. Stop Wireshark capture after execution.

Display Filter Used

Udp

```
C:\Users\JASWANTH NARNE>nslookup google.com
Server:  UnKnown
Address:  192.168.0.1

Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4009:824::200e
          142.250.195.46


C:\Users\JASWANTH NARNE>
```



Observation

The Protocol column shows UDP packets.

No SYN, ACK, or handshake packets are present.

Source Port is randomly assigned.

Destination Port is 53, which is used for DNS.

Packet exchange is fast and minimal.

Output and Explanation

UDP is a connectionless protocol. It does not establish a session before sending data. There is no acknowledgment or retransmission mechanism. This makes UDP faster than TCP, but less reliable. DNS uses UDP because it prioritizes speed over reliability.

3. ICMP Traffic Analysis

How to Generate ICMP Traffic

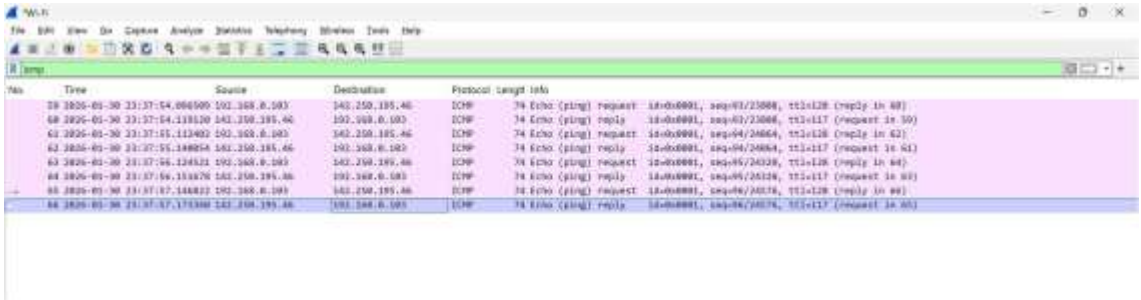
- Start Wireshark capture.
- Open Command Prompt.
- Execute the command:
- ping google.com

```
C:\Users\JASWANTH NARNE>ping google.com

Pinging google.com [142.250.195.46] with 32 bytes of data:
Reply from 142.250.195.46: bytes=32 time=22ms TTL=117
Reply from 142.250.195.46: bytes=32 time=27ms TTL=117
Reply from 142.250.195.46: bytes=32 time=27ms TTL=117
Reply from 142.250.195.46: bytes=32 time=26ms TTL=117

Ping statistics for 142.250.195.46:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 22ms, Maximum = 27ms, Average = 25ms

C:\Users\JASWANTH NARNE>
```

- Stop Wireshark capture.

Display Filter Used

Icmp



Observation:

- **ICMP Echo Request** packets are visible.
- **ICMP Echo Reply** packets are received.
- No port numbers are present.
- Source and destination IP addresses are visible.

Output and Explanation:

ICMP is used for **network diagnostics** and error reporting. The ping command sends ICMP Echo Requests to check whether a host is reachable. ICMP does not use ports and does not carry application data.

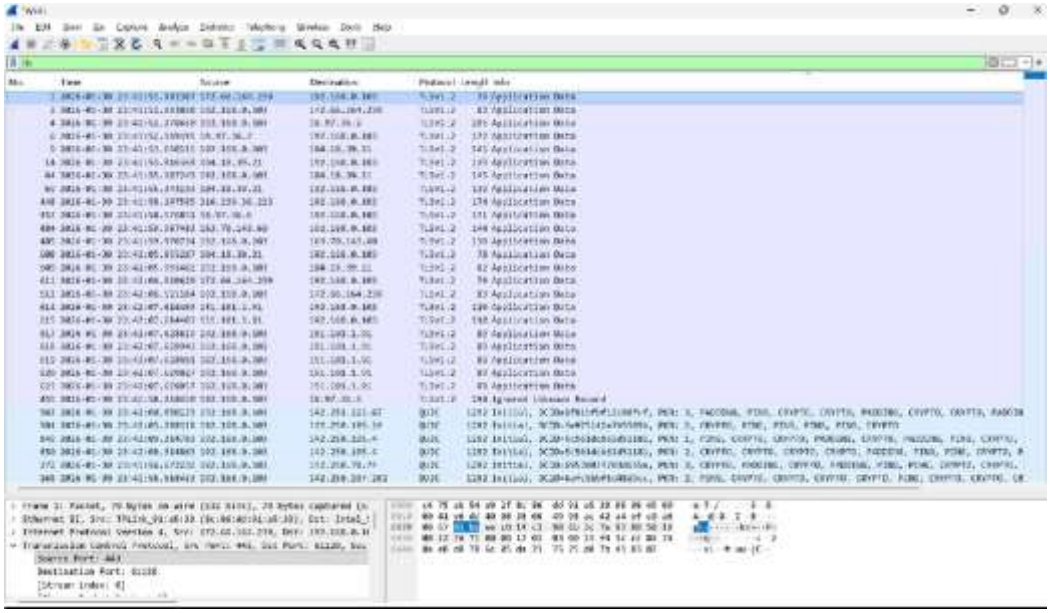
4. HTTPS Traffic Analysis

How to Generate HTTPS Traffic

- Open a web browser and visit:
- https://google.com

Display Filter Used

Tls



Observation

- Packets are encrypted and unreadable.
- TLS handshake packets are visible.
- No plaintext data is visible.
- **Destination Port** is **443**.

Output and Explanation

HTTPS uses **TLS (Transport Layer Security)** to encrypt communication between the client and server. Even though packets can be captured, the content cannot be read. This ensures confidentiality and protects sensitive data from attackers.

5. DNS Traffic Analysis

How to Generate DNS Traffic

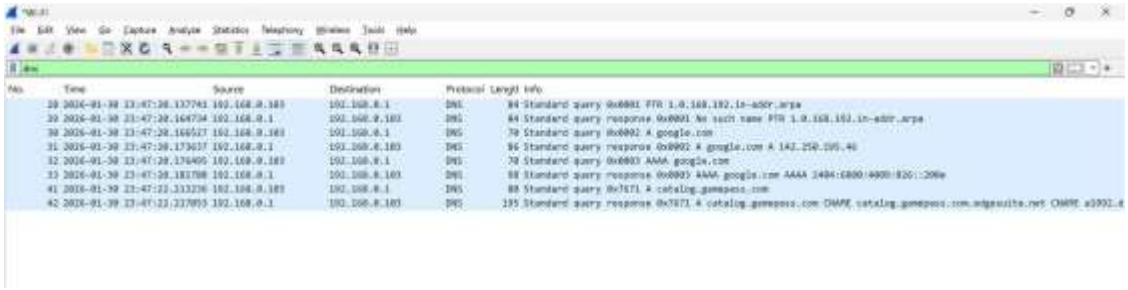
- Start Wireshark capture.
- Run the command:
- nslookup google.com

```
C:\Users\JASWANTH NARNE>ping google.com

Pinging google.com [142.250.195.46] with 32 bytes of data:
Reply from 142.250.195.46: bytes=32 time=22ms TTL=117
Reply from 142.250.195.46: bytes=32 time=27ms TTL=117
Reply from 142.250.195.46: bytes=32 time=27ms TTL=117
Reply from 142.250.195.46: bytes=32 time=26ms TTL=117

Ping statistics for 142.250.195.46:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 22ms, Maximum = 27ms, Average = 25ms

C:\Users\JASWANTH NARNE>
```

- Stop Wireshark capture.

Display Filter Used

Dns



<u>Observation</u>

DNS Query packet contains the domain name.

DNS Response packet contains the resolved IP address.

DNS typically uses UDP.

Destination Port is 53.

<u>Output and Explanation</u>

DNS translates human-readable domain names into IP addresses. Without DNS, users would need to remember IP addresses for every website. DNS traffic is usually quick and lightweight, which is why it uses UDP.

Conclusion

This project successfully demonstrates live network traffic capture and protocol analysis using Wireshark. By analyzing TCP, UDP, ICMP, HTTPS, and DNS traffic, the behavior of different protocols was clearly understood. The experiment highlights how reliable and unreliable protocols differ, how encryption protects data, and how basic network communication occurs in real-world environments.